

Lab Thread

2025/4/10 王天择

wtz514@sjtu.edu.cn

Lab内容

- **Task 1** (*uthread*). 编写一个用户态的多线程库，在 xv6 内实现。
 - 首先了解xv6是如何切换线程（进程）的。
 - 编写/修改内核代码，支持用户态的线程切换。
- **Task 2** (*pthread*). 用 *pthread* 编写多线程程序，并用 *pthread* 提供的互斥锁保证多线程程序的正确性，在 OS 上实现。
- **Task 3** (*barrier*). 用 pthread 提供的条件变量编写同步原语，在 OS 上实现。

POSIX 线程 pthread 和用户级线程 uthread

- 实现层次/调度方式:

- *Pthread*: 在操作系统内核级别实现, 线程的创建、销毁和切换等操作由操作系统内核直接管理和调度。
- *Uthread*: 在用户空间实现的线程库 (协程), 线程的管理和调度完全由用户空间的线程库进行。

- 开销:

- *Pthread* 线程的创建、销毁和切换等操作会涉及系统调用, 因此会有一定的开销。
- *Uthread* 通常比 *pthread* 更轻量级, 开销更小。

- 可移植性:

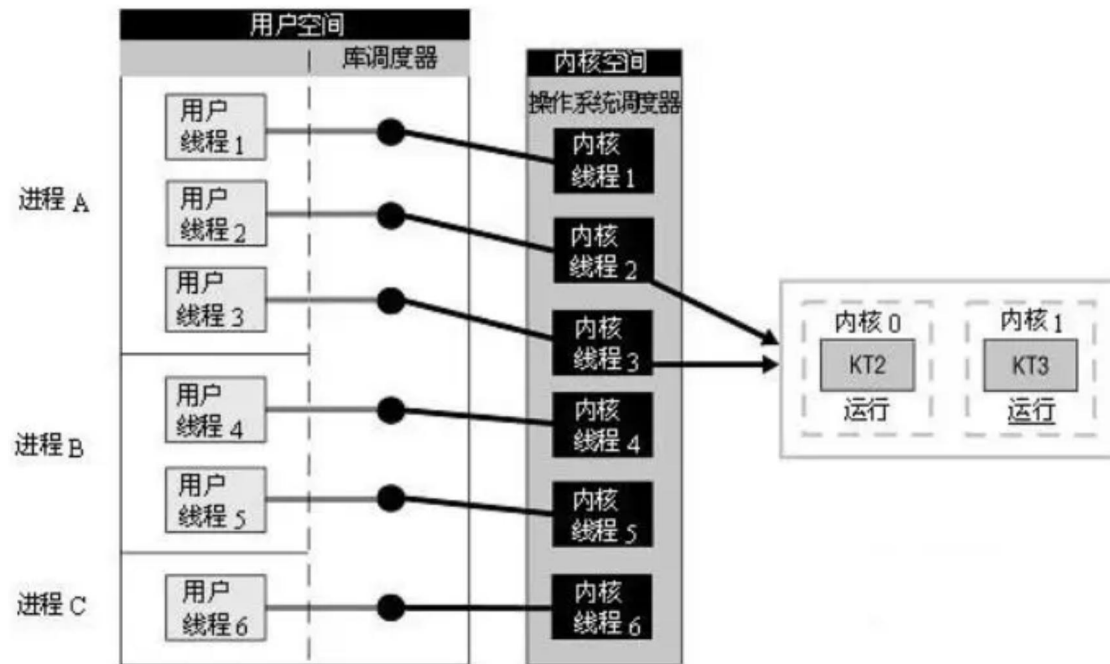
- *Pthread* 是 POSIX 标准的一部分, 因此在符合 POSIX 标准的操作系统上都可以使用 *pthread*。
- *Uthread* 则是特定于某个用户空间线程库的实现, 通常不具备跨平台的特性, 且需要特定的库支持。

进程切换

- **线程状态**包含三个部分：
 - 程序计数器 (program counter)。表示当前线程执行指令的位置。
 - 保存变量的寄存器。
 - 程序的 stack。每个线程一般都会有自己的 stack，用来记录函数调用的记录，并反映当前线程的执行点。
- **多线程并行运行的策略**：
 - 策略一：多核处理器上使用多个 CPU，每个 CPU 都可以运行一个线程。
 - 策略二：一个 CPU 在多个线程之间进行线程切换。
- 多线程 + 相同地址空间；每一个线程有一个独立的地址空间

xv6中的进程模型

- xv6中一个进程由数据结构表示 `struct proc` (`kernel/proc.h`)
- 所有进程存储在一个全局数组 `proc[NPROC]` (`kernel/proc.c`)
- 每个进程包含一个用户线程和一个内核线程：一对一模型
 - **用户线程**运行用户程序
 - **内核线程**处理用户线程产生的系统调用以及用户线程运行时发生的中断



xv6中的进程调度

- 两种请求可能会触发调度
 - **时钟中断**，调用 `yield()` (`kernel/trap.c`)
 - **长时间的系统调用**，例如 `sleep()`，调用 `sched()` (`kernel/proc.c`)
 - 切换进程状态
- 最终都会调用函数 `switch()`
- **在xv6里面，调度的单位是内核线程，而非用户线程**
- xv6会首先切换到该CPU的scheduler线程，然后切换到另一个进程的内核线程 (`kernel/proc.c`)
 - CPU的scheduler线程保存在 `struct cpu cpus[NCPU]` 中

准备环境

- 在Lab2的基础上直接修改即可

Uthread 实现用户态线程切换

- 两个文件: (1) `user/uthread.c` (2) `user/uthread_switch.S`
- 相关函数及功能:
 - `thread_create()`: 给线程分配栈空间, 初始化 ra 和 sp
 - `thread_schedule()`
 - 选择另一个可运行的线程运行
 - 直接切换线程, 没有 xv6 中的 scheduler thread
 - `thread_switch()`: 类似于 xv6 内核中的 `switch()`, 汇编实现

xv6中的线程切换和多线程同步

- (实验展示)

Pthread 实现多线程同步

- 两个文件: (1) `notxv6/ph.c` (2) `notxv6/barrier.c`
- 相关函数及功能:
 - `pthread_mutex_lock()`: 获取互斥锁, 阻塞其他线程访问临界区。
 - `pthread_mutex_unlock()`: 释放互斥锁, 允许其他线程对临界区进行访问。
 - `pthread_cond_broadcast()`: 用于向所有等待在特定条件变量上的线程发送信号, 通知它们条件已经发生变化, 从而唤醒它们继续执行。
 - `pthread_cond_wait()`: 将线程置于等待状态, 直到满足特定条件为止。它通常与互斥锁一起使用, 以确保线程在等待条件变量时不会错过条件的变化, 并在条件满足时被唤醒。